

# AI Engineering Challenge

## Build a real AI module — 3 days

May 2026 · Distributed by Sanket

### Overview — read this before your brief

You are building one AI module that solves a real problem in an AI-powered career development system. This is not a college assignment. Your output will be evaluated for real integration into a production codebase.

Each group gets one module. Six groups, six modules. Each module has a single clear job — one input, one output, one schema. Your task is to build a prompt chain that produces that output reliably, for any valid input, in both English and Hindi contexts.

#### Context — what system are these modules part of?

You are building AI modules for a career upskilling platform. The platform personalises learning journeys for two types of users:

User A (high\_wage): Final year engineering student targeting a software engineering job. English-first. Needs technical scenarios, interview prep, structured roadmap.

User B (low\_wage): Gig worker (delivery, customer support) trying to move into a stable salaried role. Hindi-first. Needs practical scenarios, confidence building, accessible language.

Your module must work correctly for both user types. The ICP field in inputs tells you which type you are dealing with.

### Timeline

|       |                                                                                                 |
|-------|-------------------------------------------------------------------------------------------------|
| Day 1 | Read brief. Design your system prompt. Test on 3 inputs manually. Understand the schema deeply. |
| Day 2 | Test on 10 varied inputs. Find where your prompt breaks. Fix it. Handle edge cases.             |
| Day 3 | Final code on GitHub by 6 PM. 5-minute demo video recorded. Demo call scheduled for Day 4.      |

### What to submit

- GitHub repo — clean code, README with how to run in under 2 minutes from clone
- 10 test cases with inputs and outputs — 5 ICP-A, 5 ICP-B

- 5-minute demo video — run on 2 new inputs live (one ICP-A, one ICP-B), explain your prompt decisions
- 1-page prompt defense doc — why this system prompt? what did you try first? what broke?

### Evaluation rubric — same for all groups

|                                     |                                                                                                                          |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>Schema correctness (30 pts)</b>  | Output matches required JSON schema on all 10 test cases. No missing fields, no hallucinated fields, correct data types. |
| <b>ICP differentiation (25 pts)</b> | ICP-A and ICP-B outputs are meaningfully different. Tone, language, salary labels, context must fork correctly.          |
| <b>Output quality (25 pts)</b>      | Does the output feel personal, specific, and real? Would a real user feel this was written for them?                     |
| <b>Prompt defense (20 pts)</b>      | Can you explain every prompt decision in the demo call? Live input change — you predict what breaks and why.             |

#### Selection bar

75 pts and above — Selected. Your POC integrates directly into v3.

60 to 74 pts — Conditional. Bring in for 1 more iteration week.

Below 60 pts — Not selected. Next best candidate in the group is evaluated.

Auto-fail overrides score: schema breaks on 3+ cases OR cannot defend prompt = rejected regardless of total.

#### Mock input JSONs — use these as your test inputs from Day 1

ICP-A: { icp\_type: "high\_wage", name: "Riya Sharma", current\_role: "Final year CS student", target\_role: "Software Engineer", urgency\_months: 6, skills: ["Python basics", "HTML", "SQL"], language: "en", vision\_profile: { current\_life: "Studying for placements, no internship yet", main\_blocker: "No real project experience", vision\_12mo: "Working at a product company, building real features", top\_motivation: "Want to prove the CS degree was worth it" } }

ICP-B: { icp\_type: low\_wage, name: Arjun Yadav, current\_role: Delivery partner, target\_role: Data entry executive, urgency\_months: 3, skills: [Basic smartphone use, Some Excel], language: hi, vision\_profile: { current\_life: Delivering 10 hours a day no fixed salary, main\_blocker: No computer skills no idea where to start, vision\_12mo: Office job fixed salary stability, top\_motivation: Stability for my family } }

## Group 4 — Interview Conductor + Interview Scorer

|                   |                                                                             |
|-------------------|-----------------------------------------------------------------------------|
| <b>Difficulty</b> | Hard                                                                        |
| <b>Maps to</b>    | Powers the full mock interview system — conduct the interview then score it |

### What you are building

You are building two AI modules that work as a pair. Module A (Interview Conductor) runs the live mock interview — it generates contextually appropriate questions and adapts the next question based on the previous answer. Module B (Interview Scorer) takes the full interview transcript and produces a structured rubric-based feedback report. Build and demo both as a connected pipeline.

### Why this matters in the product

A mock interview that asks fixed questions is just a quiz. The value is in adaptive questioning — if the user gives a strong answer on communication, the next question tests technical depth. If they give a weak answer, the follow-up probes the gap. Then the scorer must be honest enough to tell the user exactly where they fell short against a real hiring bar — not generic tips, specific moments.

### Input and output schema

#### INPUT

Module A (Conductor): `icp_type`, `target_role` (e.g. Software Engineer / CX Associate), `round_type` (screening / technical / behavioral), `company_tier` (startup / mid / enterprise), `language` (en / hi), `previous_qa_pairs[]` {question, answer\_transcript}.  
Module B (Scorer): `full_transcript[]` {question, answer}, `icp_type`, `target_role`, `round_type`, `hiring_bar` {communication: int, technical: int, problem\_solving: int, behavioral: int, delivery: int}

#### OUTPUT

Module A: `next_question` (string), `question_type` (technical / behavioral / follow\_up), `difficulty_level` (1-5), `reasoning` (why this question given previous answer).  
Module B: `overall_score` (0-100), `scores_per_axis` {communication, technical, problem\_solving, behavioral, delivery each 0-100}, `gap_vs_bar` {per axis: user score minus hiring bar}, `weak_moment` {timestamp\_approx, quote from transcript, why it hurt}, `strong_moment` {quote, why it helped}, `next_action` (1 specific drill recommendation)

### Key challenge

#### Where most people will fail on this POC

The hardest part of Module A is adaptive logic — the next question must genuinely respond to what the user said. If the user mentioned Docker in their last answer, the next technical question should probe Docker specifically. For Module B, `weak_moment` and `strong_moment` must quote the actual transcript — invented quotes are an auto-fail. `gap_vs_bar` must show negative numbers when below bar. Do not soften this.

### Test requirement

Build a mini end-to-end demo: run Module A for 3 turns of a mock screening interview (3 adaptive questions given 3 mock answers), then pass the full transcript to Module B and get the feedback report. Test for both high\_wage (SWE screening, English) and low\_wage (CX associate behavioral, Hindi). Feedback must differ between a strong and a weak candidate answer set.

### Science note — why the product is designed this way

Deliberate Practice (Ericsson) — the interview must be feedback-rich and specific to actual performance. Question difficulty should adapt to the demonstrated level. A user who answers poorly gets a simpler follow-up probing the same gap. A user who answers well gets a harder extension. Your difficulty\_level and reasoning fields in Module A must show this logic explicitly.

#### Auto-fail conditions

Module A generates the same questions regardless of previous answers. Module B weak\_moment or strong\_moment quotes are not from the actual transcript. gap\_vs\_bar shows positive numbers when user is clearly below hiring bar. Hindi output is translated English.

## Rules and important notes

### What you can use

- Anthropic Claude API — model: claude-sonnet-4-20250514
- Any language: Python, JavaScript, TypeScript
- Jupyter notebook, FastAPI, or simple script — your choice
- The mock input JSONs provided in the overview section

### What you cannot do

- Copy prompt logic from ChatGPT or other tools without understanding it
- Submit output that works on the 2 sample inputs but breaks on new ones
- Build a UI — this is a pure AI output POC, no frontend needed
- Use any model other than the one specified above

### The one thing Sanket will test in the demo call

#### The live change exercise

During the demo call, Sanket will give you a new input you have never seen before.

You will run your POC live on that input.

Then Sanket will change one field in the input and ask: what will change in the output and why?

If you cannot answer — you do not understand what you built.

This is the selection moment. The rubric score gets you to the call. This exercise tells Sanket if you can build AI systems that work in the real world.

